

武汉理工大学

CS599 期末大作业报告

法律责任初步分析 Agent

课程名称	企业级应用软件设计与开发
项目名称	法律责任初步分析 Agent
方向	方向一：Agentic AI 原生开发
学号	2025302937
姓名	宋怡康
专业	计算机技术 / 软件工程
指导教师	威欣
提交日期	2026 年 6 月 22 日

目录

目录	2
摘要	4
一、选题背景与设计思想	5
二、Specs 规格设计	6
2.1 Product Spec	6
2.2 API Spec	9
2.3 SDD 核心	11
三、系统架构与设计	13
3.1 总体架构	13
3.2 Agent 交互流程	15
3.3 数据设计	16
四、关键实现与代码展示	18
4.1 LegalAgentService	18
4.2 RAG 与知识库	20
4.3 前端交互	21
4.4 安全边界	23
五、测试与评估	24
5.1 功能测试	24
5.2 构建测试	26
5.3 Demo 评估	27
六、系统升级与扩展	28

七、课程总结	30
附录：运行方式	32
附录 A：项目文件与交付物对照	32
附录 B：关键术语说明	33

摘要

本项目构建了一个面向普通用户的法律责任初步分析 Agent。系统聚焦劳动纠纷、房屋租赁、民间借贷和消费维权等高频民事场景，提供从自然语言咨询到结构化法律分析再到 Markdown 报告生成的完整闭环。后端采用 Java 21 和 Spring Boot，前端采用 Vue 3 和 TypeScript，数据层使用 PostgreSQL、pgvector 和 Redis。Agent 当前以规则编排实现可解释的 MVP，同时预留 LLM Provider、Prompt 模板、向量检索和模型调用日志等扩展点。

与普通聊天机器人不同，本系统强调“可追问、可解释、可沉淀”。用户的每一轮补充信息都会进入同一个会话上下文，Agent 根据新增事实重新评估事实完整度、请求成熟度和证据强度，并把最终分析沉淀为结构化面板和报告。这种设计更适合企业级应用软件课程中要求的 Agentic AI 开发方式：不是只生成一段回答，而是围绕业务对象、状态转移、工具调用和交付物形成完整产品。

项目最终形成了前端、后端、数据层、Agent 编排和报告文档的完整工程包。前端负责多轮交互和结构化可视化，后端负责认证、会话、消息、分析和报告生成，数据层保存用户、会话、法条、类案、分析结果和报告。Agent 部分以可解释规则为主，同时保留 LLM Provider、Prompt 模板和向量检索扩展点，使系统既能在无外部模型环境下稳定演示，也能在后续接入真实大模型后继续演进。

为了便于评阅，本报告也按照课程评分点组织材料。选题部分说明问题价值和技术路线；Specs 部分说明 Product Spec、API Spec 和 SDD 核心；架构部分用图和文字说明系统分层、Agent 交互流程和数据设计；实现部分说明 Agent 编排、RAG、前端交互和安全边界；测试部分说明功能测试、构建测试和 Demo 评估；扩展部分说明真实 LLM、Agentic RAG、文件解析、LLMOps 和生产部署；总结部分则反思课程收获与不足。

一、选题背景与设计思想

普通用户遇到法律争议时，往往无法区分“事实是否完整”“证据是否足够”“应该先投诉、仲裁还是起诉”。直接让大模型回答“能不能赢”容易产生过度确定、忽略证据链和法律边界的问题。因此，本项目把目标定义为法律权益维护辅助，而不是 AI 法官或 AI 律师。

系统的设计思想是把用户描述拆解为一条 Agent 工作链路：用户描述 -> 隐私脱敏 -> 案件分类 -> 事实抽取 -> 完整度判断 -> RAG 检索 -> 责任分析 -> 证据建议 -> 风险提示 -> 报告生成。Agent 优先帮助用户补齐事实和证据，再输出初步分析，所有结果都带有免责声明和风险提示。

项目价值主要体现在三点：第一，降低普通用户整理法律材料的门槛；第二，把法律分析从模板问答升级为结构化、多步骤、可解释的 Agent 流程；第三，为后续接入真实 LLM、Agentic RAG、OCR 证据抽取和律师转接留下工程接口。

选题时我重点考虑了两个约束。第一，法律场景属于高风险信息服务，不能只追求回答流畅，而必须把不确定性、证据不足和安全边界放在产品核心位置。第二，课程要求强调企业级应用软件设计与 Agentic AI，因此项目不能停留在一个单文件 Demo，而应有登录认证、数据持久化、API 设计、前端状态管理、部署脚本和测试用例。最终选择法律责任初步分析，是因为它既有真实应用价值，又天然适合用 Agent 的多步骤推理来表达。

在具体设计上，我没有把系统设计成“法律问答百科”，而是设计成“案件整理助手”。二者差异在于，百科式问答通常围绕一个问题输出知识解释，而案件整理助手需要持续追问事实、判断证据链强弱、提示维权路径，并在最后形成一份可复核的材料。这个定位也决定了系统必须支持会话状态、结构化结果、证据清单和报告生成，而不是只展示一段大模型文本。

项目的核心原则包括四点：事实优先，所有判断都围绕用户已经提供的事实；证据优先，优先提醒用户补强工资流水、聊天记录、合同、订单等证据；边界优先，不

承诺胜诉、不输出正式法律意见；可扩展优先，把分类、抽取、检索、推理和报告生成拆成可替换模块，为后续 LLM 和 RAG 升级留下空间。

二、Specs 规格设计

2.1 Product Spec

系统服务对象为普通民事争议用户，核心功能包括账号登录、法律咨询会话、SSE 流式回复、结构化案件分析、法条/类案检索、证据建议、风险提示和报告生成。

从产品角度看，系统把用户咨询拆成三个层次：第一层是面向用户的自然语言交互，保证用户可以像描述真实经历一样输入材料；第二层是面向 Agent 的结构化中间状态，包括案件类型、事实字段、证据线索、争点和风险；第三层是面向交付的报告和后续维权路径，让用户可以把咨询结果带到劳动监察、仲裁、平台投诉或律师沟通场景中继续使用。

用户故事可以概括为：用户登录系统后创建一个法律咨询会话，输入“公司拖欠工资三个月且未签劳动合同”一类自然语言描述；系统先给出初步结论和缺失信息追问；用户继续补充工资标准、欠薪月份、考勤和工资流水等事实；Agent 重新计算事实完整度并更新争点、证据强度和维权路径；最后用户点击“生成报告”，获得一份可下载的案件分析材料。这个流程覆盖了课程要求中的 Agent 交互、工具调用、结构化输出和最终交付。

为了保证 MVP 范围可控，系统先聚焦四类高频民事场景，而没有泛化到全部法律领域。这四类案件共同特点是：事实要素相对稳定、证据类型可枚举、普通用户需求高、适合通过多轮追问提升材料完整度。例如劳动纠纷关注入职时间、工资标准、欠薪月份、劳动关系证据；民间借贷关注借款金额、交付方式、还款期限、借贷合意和对方身份。通过这种领域限定，系统可以在课程周期内做出较完整的端到端体验。

当前支持四类场景：

场景	子问题示例	Agent 输出重点
劳动纠纷	欠薪、未签合同、辞退	劳动关系、工资标准、仲裁时效、劳动证据
房屋租赁	押金不退、提前退租、维修	合同条款、押金金额、交接记录、扣款依据
民间借贷	无借条、转账、利息	借贷合意、款项交付、身份信息、还款期限
消费维权	质量问题、虚假宣传、退款	订单支付、瑕疵证据、售后记录、投诉路径

为了让需求更容易验收，我将核心能力整理为需求追踪矩阵。该矩阵把课程要求、用户需求和代码实现连接起来，评阅者可以从功能目标直接定位到系统中的实现模块。

需求编号	需求描述	主要实现模块	验收方式
FR-01	用户可注册、登录并维护会话状态	`AuthController`, `JwtService`, `LegalSessionService`	使用测试账号登录后创建会话
FR-02	用户可进行多轮法律咨询	`ChatController`, `ChatService`, `SessionList`	连续输入两轮事实，观察回复更新
FR-03	Agent 可识别案件类型和子类型	`LegalAgentService`	输入劳动、借贷、租赁、消费案例
FR-04	Agent 可输出缺失信息追问	`CompletenessNode` 风格规则	输入事实不足案例，检查追问
FR-05	Agent 可进行争点和	`IssueAnalysis`,	查看右侧分析面板

	证据分析	`EvidenceAssessment`	
FR-06	系统可生成报告	`ReportService`, `ReportController`	点击“生成报告”并下载
FR-07	系统具备 RAG 扩展基础	`RagSearchService`, `legal_articles`, `legal_cases`	检查来源引用和数据库 schema
FR-08	系统具备安全边界	`PrivacyMaskService`, disclaimer 配置	输入敏感信息并检查脱敏和免责声明

非功能需求方面，本项目重点考虑可运行性、可解释性、可扩展性和安全性。可运行性要求项目在没有真实 LLM Key 的情况下仍可演示，因此保留 Mock/规则分析路径；可解释性要求输出不能只有结论，还要有事实、证据、争点和风险；可扩展性要求分类、抽取、检索、推理等节点可以逐步替换为模型或工具调用；安全性要求不提交密钥、不保存明文敏感信息、不输出越界法律建议。

非功能需求	设计措施	当前达成情况
可复现	README、Docker Compose、轻量 Demo 脚本	已提供三种启动路径
可解释	结构化 AgentResult 和分析面板	已展示案件类型、证据、争点
可扩展	LLM Provider、Prompt 模板、pgvector 字段	已预留接口和表结构
安全性	隐私脱敏、免责声明、风险提示	已在回复和报告中体现
可维护	前后端分层、统一 API、文档	已补充规格和架构文档

	化	
--	---	--

2.2 API Spec

系统采用 REST API 和 SSE。核心接口包括：

模块	接口
Auth	`POST /api/auth/register`, `POST /api/auth/login`
会话	`POST /api/legal-sessions`, `GET /api/legal-sessions`
聊天	`POST /api/chat/messages`, `GET /api/chat/stream/{sessionId}`
分析	`GET /api/case-analyses/{sessionId}`
报告	`POST /api/reports/{sessionId}`, `GET /api/reports/{id}/download`
知识库	`GET /api/legal-articles/{id}`, `GET /api/legal-cases/similar`

统一响应格式为：

```
{ "code": 0, "message": "ok", "data": {} }
```

API 设计遵循前后端分离和职责清晰原则。认证接口只负责用户身份和 token，法律会话接口只负责会话生命周期，聊天接口负责消息写入和流式回复，分析接口负

负责查询最新结构化结果，报告接口负责把分析结果固化为 Markdown 报告。这样的划分避免了一个接口承担过多职责，也方便后续为移动端、后台管理端或第三方集成提供相同能力。

SSE 被用于聊天回复流式展示。相较于普通 HTTP 一次性返回，SSE 可以让前端更接近真实 Agent 交互体验：用户提交后可以看到系统逐步返回分析内容，而不是长时间等待。对于法律咨询场景，流式输出还能降低等待焦虑，使“初步结论、为什么、还需确认、下一步”这些结构逐段呈现。

错误处理方面，接口统一使用 `code`、`message`、`data` 的响应格式。业务错误例如参数缺失、资源不存在、未登录、权限不足，会分别映射为明确的状态码和错误信息。统一响应格式的好处是前端可以用一套拦截器处理登录失效、提示错误和更新页面状态，降低重复代码。

典型请求示例如下。用户先创建会话，然后通过聊天接口发送消息，最后通过分析接口或报告接口获取结构化结果。

```
POST /api/legal-sessions
{
  "title": "劳动纠纷咨询"
}
```

```
POST /api/chat/messages
{
  "sessionId": 1,
  "content": "公司拖欠工资三个月且未签劳动合同，我有工资流水和考勤记录。"
}
```

```
GET /api/case-analyses/1
{
```

```
"caseType": "劳动纠纷",  
"subType": "拖欠工资 + 未签劳动合同",  
"completenessScore": 0.75,  
"conclusionLevel": "preliminary_supported"  
}
```

SSE 事件设计保持简单，主要包含 `message` 和 `done` 两类事件。`message` 用于持续推送 Agent 回复片段，`done` 用于通知前端本轮生成结束并刷新结构化面板。这样既满足流式体验，又避免引入过重的 WebSocket 状态管理。

2.3 SDD 核心

Agent 输出对象包含案件类型、子类型、诉求目标、完整度评分、结论等级、事实、缺失问题、争点、证据评估、行动路径、风险、引用来源和用户回复。这种结构化设计让前端展示、报告生成和自动化测试复用同一份结果。

结构化输出还承担了“防幻觉”的作用。系统不会直接把模型自然语言回复当作唯一结果，而是把关键判断拆成可检查字段：哪些事实已经确认、哪些证据已经出现、哪些请求目前可主张、哪些事项仍需补充。前端分析面板、报告生成和测试断言都围绕这些字段工作，从而降低回答漂移和不可验证输出的风险。

从软件设计文档角度看，`AgentResult` 是系统最重要的数据契约。它既是 Agent 内部推理的终点，也是前端分析面板、报告服务和自动化测试的输入。只要这个契约稳定，后续无论把内部节点换成规则、LLM Function Calling 还是 LangGraph 风格状态机，外部模块都不需要大改。这体现了企业级应用中“内部实现可演进、外部接口保持稳定”的设计思路。

结论等级采用 `needs_more_facts`、`preliminary_possible`、`preliminary_supported` 等分级，而不是简单输出“能”或“不能”。这样做的原因是法律问题通常依赖证据和程序条件，过早给出确定结论容易误导用户。结论等级可

以把系统的信心和事实完整度显式表达出来，让用户知道当前回答是“需要补充事实”“有初步方向”还是“现有材料较支持”。

AgentResult 的字段设计如下：

字段	含义	前端/报告用途
`caseType`	案件大类	展示标签、决定分析模板
`subType`	子类型	展示更细分的纠纷方向
`claimGoals`	用户可能诉求	生成诉求目标和请求成熟度
`completenessScore`	事实完整度	决定追问还是分析
`conclusionLevel`	初步结论等级	控制输出语气和风险提示
`facts`	结构化事实	驱动分析面板和报告
`missingQuestions`	缺失信息问题	引导用户补充事实
`issues`	争点列表	展示法律争点和适用分析
`evidenceAssessments`	证据评估	展示已出现线索和待补材料
`actionPath`	行动路径	给出投诉、仲裁、诉讼等建议
`risks`	风险提示	提醒时效、证据灭失、送达风险
`citations`	参考来源	支持 RAG 来源说明

这个数据契约使系统能够同时满足“可展示”和“可评估”。前端可以把字段拆成不同卡片，报告服务可以把字段转成 Markdown 段落，测试用例也可以直接断言某些字段是否存在。相比只保存一段自然语言回复，这种设计更适合长期迭代。

三、系统架构与设计

3.1 总体架构

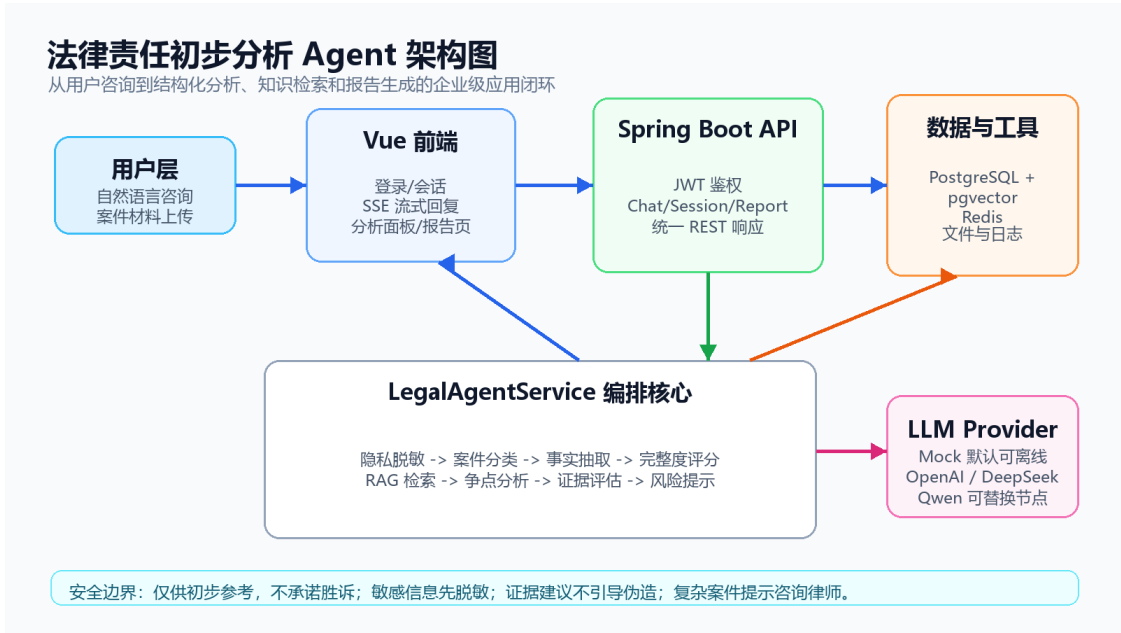


图 1：法律责任初步分析 Agent 系统架构图



整体架构采用前端、API、Agent 服务、数据存储四层划分。前端只关心页面交互和状态展示，不直接处理法律规则；API 层负责鉴权、会话、消息、报告等业务入口；Agent 服务层负责具体分析逻辑；数据层负责持久化用户、会话、消息、知识库和分

析结果。这种分层可以让项目在规模变大时保持清晰边界，例如后续要替换前端框架或接入新的模型服务时，不需要重写数据库结构和核心业务接口。

后端选择 Spring Boot 是因为它适合构建课程要求中的企业级应用：依赖管理成熟，安全、配置、测试、Web API、数据库访问都有稳定生态。前端选择 Vue 3 和 Element Plus，是因为它能快速构建管理后台式界面，适合展示会话列表、聊天窗口、分析面板、报告按钮等业务组件。数据层使用 PostgreSQL 是为了保证结构化数据可靠存储，预留 pgvector 则用于后续把法条和案例升级为向量检索。

Redis 在当前 MVP 中主要作为企业级架构预留，用于后续缓存会话状态、限流、任务队列或模型调用中间结果。虽然当前项目规模不大，但在架构图和配置中保留 Redis，体现了系统向生产环境演进时对性能和状态管理的考虑。

各层之间的数据流如下：用户在前端输入案件描述后，前端通过 API 将消息提交到后端；后端保存用户消息，并调用 Agent 服务生成结构化结果；Agent 服务根据案件类型调用 RAG 检索，合并规则分析、证据评估和风险提示；ChatService 将回复写入消息表，并通过 SSE 推送给前端；ReportService 在用户点击生成报告时读取最新分析结果，生成 Markdown 报告并保存。这个流程保证了聊天、分析和报告共享同一份后端状态。

部署方面，[docker-compose.yml](#) 将 PostgreSQL、Redis、后端和前端放在同一个编排文件中。数据库使用 [pgvector/pgvector:pg16](#) 镜像，避免单独安装向量扩展；后端使用 Maven + Temurin 21 镜像，保证 Java 版本一致；前端使用 Node 镜像运行 Vite 开发服务。这样的部署方式虽然还不是生产级 Kubernetes，但已经满足课程项目中“一键运行、依赖清晰、环境可复现”的要求。

3.2 Agent 交互流程

Agent 执行流程图

一次法律咨询从输入、追问、检索、推理到报告沉淀的运行轨迹



图 2：Agent 从用户输入到报告生成的执行流程图

```
stateDiagram-v2
    [*] --> Input
    Input --> PrivacyMask
    PrivacyMask --> CaseClassify
    CaseClassify --> FactExtract
    FactExtract --> Completeness
    Completeness --> AskMore: 信息不足
    AskMore --> Input
    Completeness --> RagSearch: 可进入分析
    RagSearch --> Reasoning
    Reasoning --> Evidence
    Evidence --> Risk
    Risk --> StructuredOutput
    StructuredOutput --> Report
```

Agent 交互流程体现了“状态驱动”的设计。用户输入不是直接交给模型生成答案，而是先进入脱敏和分类，再根据案件类型套用不同的事实字段和证据规则。事实完整度不足时，系统会优先生成追问；事实达到最低分析阈值后，才进入 RAG 检索、争点推理、证据评估和风险提示。这个流程可以避免用户只输入一句模糊描述时系统过度推断。

流程中最关键的节点是完整度判断。不同案件类型的必要事实不同，例如劳动纠纷需要入职时间、用人单位、工资标准、欠薪期间和劳动关系证据；民间借贷需要借款金额、交付方式、还款期限、借贷合意和对方身份。系统通过必备事实字段计算完整度，并决定是追问还是继续分析。这样 Agent 的行为不只是“看起来会聊天”，而是真正受业务规则约束。

RAG 检索节点目前采用轻量实现，但在架构上已经承担了来源约束作用。系统会从法条和类案种子数据中召回候选来源，并把来源写入结构化结果。后续接入 embedding 后，可以把这个节点升级为关键词召回、向量召回、重排和引用校验的组合流程，使法律分析更具可追溯性。

3.3 数据设计

数据库表包括用户、会话、消息、案件分析、法条、类案、报告、文件、Prompt 模板、模型调用日志和脱敏日志。`legal_articles`、`legal_cases` 和 `legal_documents` 均预留 `embedding vector(1536)` 字段，为后续向量检索和 Agentic RAG 做准备。

数据设计围绕两个目标展开：一是支持用户会话长期保存，二是支持 Agent 分析结果可复用。`legal_sessions` 和 `chat_messages` 保存会话与消息，保证用户可以回到历史咨询继续补充事实；`case_analyses` 保存最新结构化分析结果，让前端分析面板和报告服务不需要重新解析聊天文本；`analysis_reports` 保存生成后的报告内容，保证交付物可下载、可归档。

知识库相关表分为法条、类案和文档切片三类。法条表适合保存具有明确来源和有效性的法律条文，类案表适合保存相似案件摘要，文档切片表则预留给后续更大规

模的知识库导入。三类表都保留 `source_url` 或 `metadata`，是为了让系统在引用时能够说明来源，而不是只给出没有出处的法律观点。

Prompt 模板、模型调用日志和脱敏日志目前属于预留能力，但它们对企业级 Agent 很重要。Prompt 模板可以让管理员迭代不同节点的提示词；模型调用日志可以记录 provider、token、延迟和错误，便于成本和质量评估；脱敏日志可以在不保存原始敏感信息的前提下支持审计和问题排查。

主要数据表说明如下：

数据表	关键字段	设计说明
<code>`users`</code>	<code>`username`</code> , <code>`password_hash`</code> , <code>`role_code`</code>	保存用户身份和角色
<code>`legal_sessions`</code>	<code>`user_id`</code> , <code>`title`</code> , <code>`case_type`</code> , <code>`status`</code>	保存每个法律咨询会话
<code>`chat_messages`</code>	<code>`session_id`</code> , <code>`role`</code> , <code>`content`</code> , <code>`metadata`</code>	保存用户与 Agent 消息
<code>`case_analyses`</code>	<code>`facts_json`</code> , <code>`issues_json`</code> , <code>`risks_json`</code>	保存结构化分析结果
<code>`legal_articles`</code>	<code>`title`</code> , <code>`content`</code> , <code>`source_url`</code> , <code>`embedding`</code>	保存法条知识和向量字段
<code>`legal_cases`</code>	<code>`case_type`</code> , <code>`summary`</code> , <code>`source_url`</code> , <code>`embedding`</code>	保存类案摘要和向量字段
<code>`analysis_reports`</code>	<code>`session_id`</code> , <code>`content_md`</code>	保存生成后的 Markdown 报告
<code>`prompt_templates`</code>	<code>`code`</code> , <code>`content`</code> , <code>`enabled`</code>	预留 Prompt 动态管理

<code>`model_call_logs`</code>	<code>`provider`, `latency_ms`, `error_message`</code>	预留模型调用观测
--------------------------------	--	----------

其中 `case_analyses` 是连接 Agent 和前端的关键表。它保存的不是纯文本，而是 JSON 化的事实、争点、证据、路径和风险。这样即使用户刷新页面或重新进入会话，前端也能直接恢复分析面板，不需要重新触发一次模型调用或规则分析。

四、关键实现与代码展示

4.1 LegalAgentService

`LegalAgentService` 是核心 Agent 编排服务。它接收用户文本后，依次完成脱敏、案件识别、事实抽取、完整度计算、追问生成、争点分析、证据评估、RAG 检索、行动路径和风险提示，最终返回 `AgentResult`。

关键实现特点：

- 通过关键词和规则识别劳动纠纷、房屋租赁、民间借贷、消费维权。
- 每类案件配置不同的必要事实和证据规则。
- 使用完整度评分控制“先追问还是先分析”。
- 输出 `needs_more_facts`、`preliminary_possible`、`preliminary_supported` 等结论等级。
- 在回复中统一加入“仅供初步参考，不构成正式法律意见”。

在实现上，`LegalAgentService` 并不是简单的 if-else 问答模板，而是把案件处理拆成多个内部步骤。第一步根据关键词识别案件大类和子类型；第二步从文本中抽取金额、日期、地点、合同、聊天记录、支付记录、照片视频等事实线索；第三步根据不同案件类型的必要字段计算完整度；第四步生成缺失问题；第五步基于案件类型生成争点、证据清单、行动路径和风险提示。每一步都有明确输入和输出，便于单元测试和后续替换。

以劳动纠纷为例，系统会关注“是否能够证明劳动关系”“拖欠工资责任”“未签书面劳动合同责任”“仲裁时效和程序前置风险”等争点。对于每个争点，系统会分别整理已知事实、缺失事实、法律规则、事实适用和初步结论。这样的结构接近法律分析中的“规则、事实、适用、结论”框架，比直接输出一段建议更容易检查和复用。

规则实现还有一个优点：即使没有外部大模型 API Key，系统仍然可以稳定完成课堂演示。这对课程项目很重要，因为答辩或验收环境可能没有可用网络或模型额度。后续如果需要增强表达能力，可以只把分类、事实抽取或推理节点替换成 LLM 调用，而不必推翻整体系统。

核心节点职责可以进一步拆解为：

节点	主要输入	主要输出	当前实现方式
隐私脱敏	原始用户文本	脱敏文本	正则与替换规则
案件识别	脱敏文本	案件类型、子类型	关键词评分
事实抽取	文本、案件类型	金额、日期、证据等事实	正则和关键词
完整度评估	事实字段	分数、缺失字段	必备字段覆盖率
追问生成	缺失字段	用户追问列表	案件类型模板
争点分析	事实、案件类型	争点、规则、适用	规则化法律分析
证据评估	事实、证据规则	已有证据、待补证据	证据关键词匹配
风险提示	事实、争点	时效和程序风险	风险规则
结构化输出	全部中间结果	AgentResult	统一数据契约

例如在劳动纠纷中，用户输入“工资流水”和“考勤打卡”，证据评估节点会把工资流水识别为证明工资标准和欠薪金额的强证据，把考勤记录识别为证明实际提供劳动的中等或强证据；如果用户没有提供入职时间或实际工作地，完整度节点会生成对应追问。这种行为使系统更像一个按法律事实清单工作的 Agent，而不是普通关键词问答。

4.2 RAG 与知识库

`RagSearchService` 从法条和类案表中检索候选来源。当前 MVP 使用轻量关键词检索保证可离线运行，数据库 schema 预留 `pgvector` 字段，后续可升级为 `embedding` 召回、重排和来源约束生成。

当前知识库中预置了劳动合同法、劳动争议调解仲裁法、民法典合同编、消费者权益保护法等基础材料，以及劳动纠纷、租赁纠纷、民间借贷、消费维权的示例案例。虽然数据规模不大，但足以展示 RAG 的基本思想：Agent 不仅依赖内部规则，还可以把检索到的法条和类案作为参考来源加入回复和报告。

RAG 节点在当前版本中承担三个职责。第一是根据案件类型缩小检索范围，避免劳动纠纷引用消费维权材料；第二是把来源标题和 URL 写入 `citations`，便于报告中展示参考依据；第三是为后续 `embedding` 检索提供结构化表和服务接口。后续如果引入向量模型，只需要为 `legal_articles` 和 `legal_cases` 写入 `embedding`，再扩展检索排序逻辑。

为了避免来源滥用，系统设计上要求 RAG 结果只是“参考来源”，不能把检索结果直接等同于最终法律结论。Agent 仍需结合用户事实和证据强度进行判断。当检索不到可靠来源时，系统可以输出一般性提示，但应降低结论强度，并提醒用户咨询专业律师。

知识库导入和治理的后续流程可以设计为：管理员收集法条或案例来源，系统进行文档切片和 `metadata` 标注，生成 `embedding` 后写入 `legal_documents`，检索时先

按案件类型和有效性过滤，再进行关键词召回和向量召回，最后把候选结果交给重排节点筛选。这样可以避免把过期法条、不相关案例或来源不明内容直接暴露给用户。

RAG 对法律场景尤其重要，因为法律建议需要可追溯来源。即使当前版本只是轻量实现，也已经在数据结构和报告内容中保留“参考依据”位置。后续一旦补充真实知识库，报告就可以显示具体法条标题、来源 URL、案例摘要和相关度分数，提高可信度和评阅效果。

4.3 前端交互

前端使用 Vue 3、Pinia 和 Element Plus。用户登录后进入聊天页面，系统通过 SSE 展示流式 Agent 回复，并在结构化分析面板中展示案件类型、缺失问题、证据、风险和引用来源。报告页面调用后端接口生成 Markdown 报告。



图 3：法律责任分析 Agent 实际执行界面

实际界面采用三栏结构：左侧是法律咨询会话列表，中间是多轮对话窗口，右侧是实时案件分析面板。中间对话区展示 Agent 的初步结论、原因说明、仍需确认的信息和下一步建议；右侧面板同步展示案件分类、诉求目标、请求成熟度、缺失信息、

争点分析和参考依据。用户每补充一次事实，Agent 都会重新整理上下文，使输出从“泛泛建议”逐步收敛为“围绕本案事实和证据的可操作分析”。

界面右上角提供在线生成报告入口。该按钮不是简单截图导出，而是调用后端报告服务，基于最新的 `case_analyses` 结构化结果生成 Markdown 报告。这样可以保证报告内容与 Agent 面板中的案件类型、证据强度、风险提示保持一致，也方便后续扩展为 PDF/Docx 导出。

前端状态管理使用 Pinia，把认证状态和聊天状态拆分到不同 store 中。认证 store 维护 token 和用户信息，聊天 store 维护会话列表、当前会话、消息流和分析结果。这样做可以避免组件之间直接传递大量 props，也让登录状态失效、切换会话、追加 SSE 消息等操作更容易维护。

页面交互上，系统提供了快捷输入标签，例如“欠薪未签合同”“借钱不还”“押金不退”“消费退款”等，帮助演示时快速触发典型案例。输入框支持用户继续补充事实，右侧面板则展示最新结构化结果。这个设计能体现 Agent 的多轮能力：不是一轮问答结束，而是随着事实补充持续更新分析。

报告生成功能体现了“从交互到交付”的闭环。普通聊天系统往往停留在屏幕上的回答，而本项目将回答沉淀为报告记录，便于下载、复核和课堂展示。报告服务复用后端分析结果，而不是重新让前端拼接文本，因此格式更稳定，也更适合后续生成 PDF 或 Docx。

界面设计上，我刻意采用更接近工作台的布局，而不是营销式首页。法律咨询场景需要长时间阅读、比较和补充材料，因此左侧会话、中间对话、右侧分析面板的布局更适合实际操作。右侧面板始终可见，可以让用户在多轮对话中持续看到案件状态变化，例如事实完整度从较低提升到较高，请求成熟度从“需要补充”变为“可主张”。

前端交互还考虑了演示效率。课堂展示时，如果每次都手动输入长案例会浪费时间，因此页面底部提供常见问题快捷按钮。用户点击“欠薪未签合同”后可以快速生

成一个典型劳动纠纷输入，再通过补充事实展示 Agent 的多轮推理能力。这种设计对真实产品也有意义，因为普通用户常常不知道如何开始描述案件，快捷入口能降低输入门槛。

4.4 安全边界

系统实现了隐私脱敏服务，避免身份证、手机号、银行卡等敏感信息直接进入分析链路。Agent 回复不承诺胜诉、不指导伪造证据，并对高风险情况提示咨询专业律师。

安全边界在本项目中不是单独的提示语，而是贯穿整个执行链路：输入阶段先做隐私脱敏，分析阶段根据事实完整度控制结论强度，输出阶段统一加入免责声明和风险提示，报告阶段保留“仅供初步参考”的定位。对于劳动仲裁时效、对方失联、证据灭失、金额较大等场景，系统会提示用户尽快固定证据或咨询律师，而不是给出绝对结论。

隐私保护方面，系统会在进入 Agent 分析前处理手机号、身份证号、银行卡号等敏感信息。这样即使后续接入外部 LLM，也可以减少敏感数据直接暴露给第三方模型的风险。当前版本还预留了脱敏日志表，可以记录脱敏类型和哈希摘要，而不保存原始敏感内容。

合规输出方面，系统避免三类危险回答：第一是不承诺“肯定胜诉”或“必然赔偿”；第二是不建议伪造、篡改、隐瞒证据；第三是不指导用户绕开法律程序。对于复杂或高风险案件，系统会将结论保持在初步分析层面，并建议用户找专业律师进一步确认。

安全控制可以归纳为以下表格：

风险	表现	控制措施
隐私泄露	用户输入手机号、身份证、银行卡	进入 Agent 前脱敏

过度承诺	输出“肯定胜诉”“一定赔偿”	统一免责声明和结论分级
证据误导	暗示伪造、隐瞒或篡改证据	风险提示中明确禁止
时效遗漏	忽略仲裁或诉讼期限	风险节点识别关键时间
来源不明	无依据引用法律规则	RAG 引用和来源字段
模型不可用	外部 LLM 无法访问	Mock/规则路径兜底

这些控制措施并不能让系统替代律师，但可以显著降低 AI 法律咨询常见的误导风险。对于课程项目而言，这也是我认为比单纯接入大模型更重要的部分：系统必须知道自己不能做什么。

五、测试与评估

5.1 功能测试

后端测试覆盖两个核心 Demo：

- 劳动纠纷：输入“公司拖欠工资三个月且未签劳动合同”，系统应识别为劳动纠纷，生成缺失问题，并包含免责声明和争点分析。
- 民间借贷：输入“朋友借钱 2 万元不还，没有借条但有微信聊天和银行转账记录”，系统应识别为民间借贷纠纷，发现借贷合意争点，并区分转账记录和借条证据状态。

测试设计重点不是覆盖所有法律情形，而是验证 Agent 的关键行为是否稳定。劳动纠纷用例验证案件分类、子类型识别、缺失问题生成和免责声明；民间借贷用例验证不同案件类型下的争点切换和证据强度判断。只要这两类代表性用例通过，就说明核心 Agent 编排没有退化为单一模板输出。

后续测试可以继续扩展四类场景的边界样本。例如劳动纠纷中加入“公司说已经结清”“员工主动离职”等对方抗辩；民间借贷中加入“对方说是赠与”“已经部分还款”；租赁纠纷中加入“提前退租”和“房东扣维修费”；消费维权中加入“超过售后期”和“商家主张人为损坏”。这些样本可以检验 Agent 是否能识别不利因素和风险。

测试用例设计表如下：

用例编号	输入摘要	预期案件类型	重点断言
TC-01	公司拖欠工资三个月，未签劳动合同	劳动纠纷	识别劳动关系、欠薪、未签合同
TC-02	朋友借钱 2 万不还，无借条但有转账	民间借贷纠纷	识别借贷合意和转账证据
TC-03	房东不退押金，称房屋损坏	房屋租赁纠纷	提示交接照片和扣款依据
TC-04	网购商品质量问题，商家拒绝退款	消费维权纠纷	提示订单、支付和瑕疵证据
TC-05	用户只说“我想起诉别人”	其他民事咨询	优先追问法律关系和事实
TC-06	输入手机号和身份证	任意	输出前敏感信息被脱敏

评价时不只看分类是否正确，还要看输出是否具备法律场景需要的保守性。例如事实不足时应追问，而不是给结论；证据不足时应提示补强，而不是假设用户有证据；涉及时效和复杂争议时应提示律师咨询，而不是直接替用户选择诉讼策略。

5.2 构建测试

后端通过 Maven 测试验证核心服务。前端通过 `npm run build` 验证 TypeScript、路由、状态管理和页面组件无编译错误。

构建测试用于保证项目可以被助教或同学拉取后复现。后端依赖 Java 21 和 Maven，前端依赖 Node.js 和 npm。Docker Compose 则提供更统一的启动方式，将 PostgreSQL、Redis、后端和前端一起编排。由于课程验收环境可能不同，README 同时提供了 Docker 启动、本地启动和轻量 Demo 三种路径，降低复现门槛。

在当前开发环境中，部分联网依赖下载可能受网络影响，因此报告和 README 都保留了可离线演示路径。`demo/run-demo.ps1` 可以启动轻量版前后端服务，用于展示核心交互。完整版本则通过 Spring Boot、PostgreSQL 和 Redis 运行，体现企业级应用结构。

构建与部署验收清单如下：

检查项	命令或方式	通过标准
后端测试	<code>`mvn test`</code>	单元测试通过
前端构建	<code>`npm run build`</code>	TypeScript 和 Vite 构建通过
Docker 启动	<code>`docker compose up`</code>	前端和后端端口可访问
轻量 Demo	<code>`demo/run-demo.ps1`</code>	页面可打开并完成对话
报告生成	点击生成报告	下载 Markdown 报告
PDF 文档	<code>`python scripts/generate-report-pdf.py`</code>	生成带书签和图片的 PDF

由于本项目最终需要提交到 GitHub，仓库检查也很重要。`.gitignore` 排除了 `node_modules`、`target`、`.env`、日志文件等不应提交的内容；`.env.example` 保留必要

配置项但不包含真实密钥；README 提供启动说明和项目状态 Checklist；

[docs/CS599_大作业报告.pdf](#) 作为最终可阅读交付物提交。

5.3 Demo 评估

课堂演示按以下路径完成：注册登录 -> 创建会话 -> 输入劳动纠纷案例 -> 查看流式回复 -> 查看结构化分析 -> 输入民间借贷案例 -> 生成报告。评估指标包括案件识别正确性、缺失问题有效性、证据建议可操作性、风险提示完整性和报告可读性。

以劳动纠纷案例为例，用户首先输入“公司拖欠工资三个月且未签劳动合同，我有工资转账记录和考勤打卡”。系统会识别为劳动纠纷，并给出初步判断：工资转账和考勤能够证明劳动关系与欠薪线索，但仍需补充入职时间、欠薪起止月份、实际工作地和是否在职等关键信息。当用户继续补充“每月 2000 元，欠薪 2 月至 6 月，最近一次催要工资时间是 6 月 15 日”后，Agent 会把请求成熟度更新为更高等级，并进一步输出拖欠工资金额、未签书面劳动合同责任、仲裁时效和证据清单。

这个 Demo 能体现 Agentic AI 的几个关键点：第一，系统不是一次性问答，而是多轮状态更新；第二，右侧分析面板不是装饰性摘要，而是由后端结构化分析结果驱动；第三，在线报告生成基于最新会话状态，能够把聊天过程沉淀为可提交、可复核的材料；第四，安全提示和不确定性说明始终保留，避免把初步分析误导为正式法律意见。

Demo 评估可以从定性和定量两个角度进行。定性上，观察系统是否能给出用户看得懂的追问、证据建议和下一步路径；定量上，可以记录案件分类是否正确、缺失问题是否覆盖关键事实、证据建议是否命中用户已提供材料、报告是否包含案件类型、争点、证据、风险和参考来源。对于课程大作业而言，这些指标比单纯追求模型回答华丽更重要。

与课程要求对应，本项目在测试评估章节中展示了功能测试、Agent 行为评估、Demo 截图和报告生成结果。虽然目前还没有完整的 LLM0ps 平台，但数据表和服务设计已经为后续记录模型调用、延迟、错误和评估分数预留位置。

如果要进一步量化 Agent 质量，可以设计如下指标：

指标	含义	评估方式
案件识别准确率	是否识别正确案件类型	人工标注样本对比
缺失问题覆盖率	是否问到关键事实	与案件事实清单对比
证据建议命中率	是否识别已有证据和待补证据	对照用户输入和证据规则
风险提示召回率	是否提示时效、失联、证据灭失等风险	风险样本集评估
安全合规通过率	是否避免绝对结论和越界建议	人工审核或规则扫描
报告完整度	报告是否包含事实、争点、证据、路径	文档结构检查

这些指标也是后续 LLM0ps 的基础。接入真实模型后，可以对比规则 Agent 和 LLM Agent 在相同样本上的表现，观察 LLM 是否提升表达质量、事实抽取能力和追问自然度，同时检查是否带来更高幻觉风险。

六、系统升级与扩展

下一阶段可以从四个方向升级：

1. 接入真实 LLM：将案件分类、事实抽取、法律推理等节点替换为 Function Calling 或结构化输出。

2. 增强 Agentic RAG: 接入 embedding, 使用 pgvector 做向量召回, 并增加来源重排和引用约束。

3. 文件证据解析: 接入 OCR、PDF/Word 解析, 自动从工资流水、聊天截图、合同中抽取证据要素。

4. 可观测与评估: 记录模型调用日志、延迟、错误、token 消耗和人工评分, 形成 LLMops 评估闭环。

如果继续推进到生产级版本, 还需要增加三类能力。第一是知识库治理能力, 包括法条有效性、案例来源可信度、知识入库审核和过期内容下线; 第二是 Agent 评估能力, 包括不同案件类型的 Benchmark、人工标注样本和自动回归测试; 第三是交付能力, 包括报告模板配置、PDF/Docx 导出、证据附件索引和律师协作入口。这些能力会让系统从课程 MVP 进一步接近真实企业级应用。

技术演进路线可以分为三个阶段。第一阶段是当前 MVP, 目标是完成全栈闭环和规则 Agent, 保证可运行、可解释、可展示。第二阶段接入真实 LLM, 把事实抽取、追问生成和法律推理改为结构化模型调用, 同时保留规则校验和安全审查。第三阶段引入 Agentic RAG、证据文件解析和 LLMops 评估, 让系统能够处理更多材料来源, 并用数据持续评估回答质量。

产品演进路线则可以围绕“用户材料管理”展开。当前系统主要处理文本描述, 后续可以让用户上传合同、工资流水、聊天截图、订单截图和催告记录, 由 OCR 和文档解析节点抽取证据要素, 并在报告中形成附件索引。这样系统不只是回答问题, 还能帮助用户组织维权材料包。

部署演进方面, 当前 Docker Compose 适合本地和课程展示。若进入真实环境, 可以拆分前端静态资源、后端 API、数据库、Redis、对象存储和模型代理服务, 并增加 HTTPS、访问日志、备份策略、限流和权限审计。对于法律相关应用, 还需要更加严格的数据安全和用户授权机制。

未来功能路线可以按优先级排序:

优先级	功能	价值
P0	真实 LLM 结构化输出	提升事实抽取和语言表达能力
P0	知识库 embedding 检索	提升来源召回质量
P1	PDF/Word/OCR 证据解析	从用户材料中自动抽取事实
P1	报告导出 PDF/Docx	提升交付体验
P1	模型调用日志和评估面板	支持成本和质量管理
P2	律师协作或人工复核入口	支持高风险案件转人工
P2	多地区法规和时效规则	提升地域适配能力

这些功能并不是一次性完成，而应遵循“先稳定闭环，再增强智能”的原则。当前项目已经完成闭环，后续每次升级都可以只替换其中一个节点，例如先把事实抽取换成 LLM，再把 RAG 检索换成向量召回，最后增加文件证据解析。这样可以控制风险，也便于持续测试。

七、课程总结

通过本项目，我对企业级应用中的 Agentic AI 有了更具体的理解。Agent 不只是调用一次大模型，而是把任务拆分为多个有状态、有边界、可测试的节点，并用工程化方式处理身份认证、数据持久化、错误兜底、隐私保护、前后端交互和部署。

本项目仍有不足：当前法律推理主要依靠规则，真实 LLM 和向量检索尚未完全接入；文件证据解析只完成了接口占位；报告导出仍以 Markdown 为主。后续如果继续迭代，我会优先补齐 Agentic RAG、真实模型调用、证据解析和可观测评估，使系统从课程 MVP 进一步接近可用产品。

课程开发过程中最大的收获，是认识到 AI 应用开发不能只关注模型本身。一个可交付系统还需要清楚的产品边界、稳定的数据结构、可复现的部署方式、可解释的输出、可测试的业务逻辑和可维护的文档。尤其在法律场景中，回答是否“像真的”并不是最重要的，是否能说明依据、暴露不确定性、提示证据缺口和避免越界才是更关键的工程目标。

另一个收获是 SDD 和规格文档对实现有实际帮助。先定义 API、数据对象和 Agent 流程，再实现代码，可以减少后期返工。例如 `AgentResult` 的结构化设计让前端面板、报告生成和单元测试都能复用同一份数据；Docker 和 README 的启动说明则保证项目不仅能在开发者电脑上运行，也能被评阅者快速理解和复现。

如果给课程提出建议，我希望后续可以增加更多 Agent 评估样例和真实企业级案例，例如如何设计 Benchmark、如何比较规则 Agent 与 LLM Agent、如何记录模型调用成本、如何处理隐私和合规审计。这些内容与本项目后续演进方向高度相关，也能帮助学生从“能调用模型”进一步走向“能交付可靠 AI 软件”。

从个人角度看，这次项目也让我更清楚地认识到文档和演示的重要性。代码实现只是作业的一部分，评阅者还需要通过 README、架构图、执行截图、PDF 书签和章节说明快速理解系统价值。后期我补充了架构图、执行流程图、真实运行界面和更完整的章节内容，就是为了让项目不只是“能运行”，也能被清楚地讲明白。

如果重新开始做这个项目，我会更早建立测试样本库和报告模板。前期主要精力放在功能跑通，后期才集中补充评估和文档，导致报告需要多次迭代。下一次开发类似 Agent 项目时，我会在第一周就定义好用户故事、验收标准、样本集、报告结构和图表风格，让工程实现和文档交付同步推进。

附录：运行方式

附录 A：项目文件与交付物对照

文件或目录	内容	对应交付价值
`README.md`	项目介绍、启动方式、技术栈、Demo 流程	让评阅者快速理解和运行项目
`backend/`	Spring Boot 后端、Agent 服务、API、数据库配置	展示企业级后端实现
`frontend/`	Vue 3 前端、会话页、分析面板、报告入口	展示用户交互和结构化可视化
`docs/CS599_大作业报告.pdf`	最终报告 PDF，包含书签和图片	课程最终文档交付
`docs/architecture.md`	架构和 Agent 流程说明	支撑系统设计评分
`docs/spec.md`	Product/API/SDD 规格	支撑 Specs 评分
`docs/demo.md`	演示脚本和测试记录	支撑测试与 Demo 评分
`docs/assets/`	架构图、执行图、真实界面截图	提升报告可读性
`scripts/generate-report-pdf.py`	生成带书签和图片的 PDF	保证文档可复现
`docker-compose.yml`	PostgreSQL、Redis、后端、前端编排	支撑部署和复现

附录 B：关键术语说明

术语	本项目中的含义
Agentic AI	不是单次问答，而是围绕目标进行状态管理、工具调用和多步骤推理的 AI 应用形态
SDD	软件设计文档，描述接口、数据结构、状态流程和实现约束
RAG	检索增强生成，通过知识库检索为分析提供来源依据
pgvector	PostgreSQL 的向量扩展，用于后续 embedding 检索
SSE	Server-Sent Events，用于后端向前端持续推送流式回复
AgentResult	本项目核心结构化输出对象，连接 Agent、前端和报告服务
完整度评分	根据案件必要事实覆盖情况计算的分析准备度
证据评估	判断用户是否已经提供关键证据，以及还需要补充哪些材料
风险提示	对时效、证据灭失、失联、执行等不利因素的提示
安全边界	系统不承诺胜诉、不替代律师、不指导伪造证据的合规约束

Docker 启动：

```
docker compose up
```

后端测试:

```
cd backend  
mvn test
```

前端构建:

```
cd frontend  
npm install  
npm run build
```